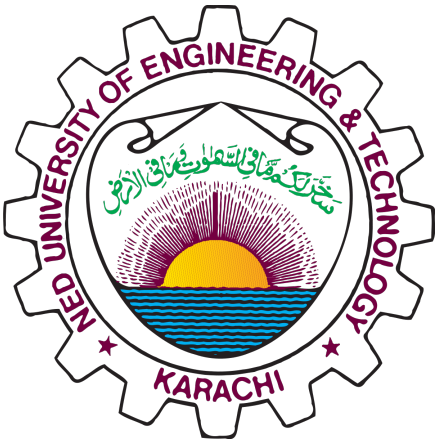


NED University
of Engineering and Technology

Team Reference Document

Team $(ttl)^3$

DP

0-1 Knapsack You are given n items with i_{th} item having weight w_i and value v_i. Given a knapsack with max Weight capacity W. Find the max value you can have.

Note that each item can only be taken **once**.

Time Complexity: $O(nW)$ **Space Complexity:** $O(W)$

```
int knapsack(int n, vector<int>& weights, vector<int>& values, int W) {
    vector<int> dp(W + 1, 0);
    for(int i = 0; i < n; i++) {
        for(int j = W; j >= weights[i]; --j) {
            dp[j] = max(dp[j], values[i] + dp[W - j]);
        }
    }
    return dp[W];
}
```

Complete Knapsack You are given n items with i_{th} item having weight w_i and value v_i. Given a knapsack with max Weight capacity W. Find the max value you can have.

Note that any item can be taken **unlimited times**.

Time Complexity: $O(nW)$ **Space Complexity:** $O(W)$

```
int knapsack(int n, vector<int>& weights, vector<int>& values, int W) {
    vector<int> dp(W + 1, 0);
    for(int i = 0; i < n; i++) {
        for(int j = weights[i]; j <= W; ++j) {
            dp[j] = max(dp[j], values[i] + dp[W - j]);
        }
    }
    return dp[W];
}
```

Multiple Knapsack You are given n items with i_{th} item having weight w_i and value v_i. Given a knapsack with max Weight capacity W. Find the max value you can have.

Note that each item can be taken k_i **times**.

Time Complexity: $O(n \sum \log k_i)$ **Space Complexity:** $O(W)$

```
// Just break each item in powers of 2 and treat each division as single item and apply 01 knapsack
```

Edit Distance Given two strings s_1 and s_2, find the minimum number of operations (insert, delete, replace) required to transform s_1 into s_2.

Time Complexity: $O(mn)$ **Space Complexity:** $O(mn)$

```
int editDistance(string& s1, string& s2) {
    int m = s1.size(), n = s2.size();
    vector<vector<int>> dp(m + 1, vector<int>(n + 1));
    for (int i = 0; i <= m; ++i) dp[i][0] = i;
    for (int j = 0; j <= n; ++j) dp[0][j] = j;
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            dp[i][j] = min(dp[i - 1][j] + 1, dp[i][j - 1] + 1, dp[i - 1][j - 1] + (s1[i - 1] != s2[j - 1]));
        }
    }
    return dp[m][n];
}
```

Strings

Polynomial Hash

Time Complexity: $O(N)$ **Space Complexity:** $O(N)$

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
const ll M = 991831889;
const ll C = uniform_int_distribution<ll>(0.1 * M, 0.9 * M)(rng);

struct HashString {
    int n;
    vector<ll> pows, sums;

    HashString(string s) : n(s.size()), pows(n + 1, 1), sums(n + 1) {
        for (int i = 1; i <= n; i++) {
            pows[i] = pows[i - 1] * C % M;
            sums[i] = (sums[i - 1] * C + s[i - 1]) % M;
        }
```

```
    }

    // Returns the hash of the substring [l, r)
    ll hash(int l, int r) {
        ll h = sums[r] - sums[l] * pows[r - l];
        return (h % M + M) % M;
    }
};
```

Trie

Time Complexity: $O(\sum |s_i|)$ **Space Complexity:** $O(26 * n)$

```
struct Vertex {
    int next[26];
    bool output = false;
    int link = -1;
    int p;
    char pch;
    int go[26];

    string text = "";
    int nearestOutput = -2;
    Vertex(int par = 0, char ch = '$'): p(par), pch(ch) {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    }
};
vector<Vertex> t(1);

int addString(string& s) {
    int v = 0;
    for(char ch: s) {
        int c = ch - 'a';
        if(t[v].next[c] == -1) {
            t[v].next[c] = t.size();
            t.emplace_back(v, ch);
        }
        v = t[v].next[c];
    }
    t[v].output = true;
    t[v].text = s;
    return v;
}

int go(int v, char ch);

int getLink(int v) {
    if(t[v].link == -1) {
        if(v == 0 || t[v].p == 0) {
            t[v].link = 0;
        } else {
            t[v].link = go(getLink(t[v].p), t[v].pch);
        }
    }
    return t[v].link;
}

int go(int v, char ch) {
    int c = ch - 'a';
    if(t[v].go[c] == -1) {
        if(t[v].next[c] != -1) {
            t[v].go[c] = t[v].next[c];
        } else {
            t[v].go[c] = v == 0 ? 0: go(getLink(v), ch);
        }
    }
    return t[v].go[c];
}

int getNearestOutput(int v) {
    if(t[v].nearestOutput == -2) {
        if(v == 0) {
            t[v].nearestOutput = -1;
        } else {
            int u = getLink(v);
            if(u == 0) {
                t[v].nearestOutput = -1;
```

```
            } else if(t[u].output) {
                t[v].nearestOutput = u;
            } else {
                t[v].nearestOutput = getNearestOutput(u);
            }
        }
    }
    return t[v].nearestOutput;
}
```

Z-Algorithm
Time Complexity: $O(N)$ **Space Complexity:** $O(N)$

```
vector<int> z_function(string& s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

Manacher Algorithm
Time Complexity: $O(N)$ **Space Complexity:** $O(N)$

```
#include "bits/stdc++.h"
using namespace std;

vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "^";
    vector<int> p(n + 2, 0);
    int l = 0, r = 1;
    for (int i = 1; i <= n; i++) {
        if (i <= r)
            p[i] = min(r - i, p[l + (r - i)]);
        while (s[i - p[i]] == s[i + p[i]])
            p[i]++;
        if (i + p[i] > r) {
            l = i - p[i];
            r = i + p[i];
        }
    }
    return vector<int>(p.begin() + 1, p.end() - 1);
}

vector<int> manacher(const string& s) {
    string t;
    t.reserve(2 * s.size() + 1);
    for (char c : s) {
        t += '#';
        t += c;
    }
    t += '#';
    return manacher_odd(t);
}

void solve()
{
    string s;
    cin >> s;
    string org = s;

    vi p = manacher(s);
    vi ans(org.size());
    vi right(org.size());
    f(i, 1, p.size() - 1) {
```

```
        int len = p[i] - 1;
        int pos = i / 2 + len / 2;
        if(i % 2 == 0) pos--;
        // Starting from pos with 'len' length palindrome
        right[pos] = max(right[pos], len);
    }

    int maxLen = 0;
    fr(i, org.size() - 1, -1) {
        maxLen = max(right[i], maxLen);
        ans[i] = maxLen;
        maxLen -= 2;
    }

    f(i, 0, org.size()) {
        cout << ans[i] << ' ';
    }

}
```

KMP KMP Automation

Time Complexity: $O(N)\&O(N.M)$ **Space Complexity:** $O(N)\&O(N.M)$

```
#include "bits/stdc++.h"

using namespace std;

void kmpAutomation(int n, string& p) {
    int m = p.size();

    // Step 1: Compute KMP pi array
    vector<int> pi(m);
    for (int i = 1; i < m; i++) {
        int j = pi[i - 1];
        while (j > 0 && p[i] != p[j])
            j = pi[j - 1];
        if (p[i] == p[j])
            j++;
        pi[i] = j;
    }

    // Step 2: Build String Automaton
    vector<vector<int>> aut(m, vector<int>(26));
    for (int j = 0; j < m; j++) {
        for (int c = 0; c < 26; c++) {
            if (j > 0 && 'A' + c != p[j])
                aut[j][c] = aut[pi[j - 1]][c];
            else
                aut[j][c] = j + ('A' + c == p[j]);
        }
    }

    // Step 3: DP Table
    // dp[i][j] = ways to build a string of length i ending in automaton state j
    vector<vector<long long>> dp(n + 1, vector<long long>(m + 1, 0));
    dp[0][0] = 1;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j <= m; j++) {
            if (dp[i][j] == 0) continue;

            if (j == m) {
                // Absorbing state: once matched, any character keeps it matched
                dp[i + 1][m] = (dp[i + 1][m] + dp[i][m] * 26LL) % MOD;
            } else {
                // Transition to next state for all 26 characters
                for (int c = 0; c < 26; c++) {
                    int nxt = aut[j][c];
                    dp[i + 1][nxt] = (dp[i + 1][nxt] + dp[i][j]) % MOD;
                }
            }
        }
    }

    cout << dp[n][m] << "\n";
}
```

DataStructures

Union Find

Time Complexity: $O(LogN)$ **Space Complexity:** $O(N)$

```
vi link(n + 1), size(n + 1);
f(i, 1, n + 1) link[i] = i;
f(i, 1, n + 1) size[i] = 1;

int find(int x) {
    if(x == link[x]) return x;
    return link[x] = find(link[x]);
}

bool same(int x, int y) {
    return find(x) == find(y);
}

void union(int a, int b) {
    a = find(a);
    b = find(b);
    if(a != b) {
        if(size[a] < size[b]) swap(a, b);
        size[a] += size[b];
        link[b] = a;
    }
}
```

Fenwick Tree

Time Complexity: $O(LogN)$ **Space Complexity:** $O(N)$

```
// Use 1 based indexing
long long sum(vll& tr, ll k) {
    ll s = 0;
    while(k >= 1) {
        s += tr[k];
        k -= k&-k;
    }
    return s;
}

void add(vll& tr, ll k, ll x) {
    while(k < tr.size()) {
        tr[k] += x;
        k += k&-k;
    }
}
```

Segment Tree

Time Complexity: $O(LogN)$ **Space Complexity:** $O(4 * N)$

```
// Set Identity / Base value returns
struct LazySegTree {
    int n;
    vector<ll> t, lz; // t = sum on segment, lz = pending add for children
    LazySegTree(int n) : n(n), t(4 * n), lz(4 * n) {}
    LazySegTree(const vector<ll>& a) : LazySegTree((int)a.size()) {
        build(1, 0, n, a);
    }
    void build(int v, int l, int r, const vector<ll>& a) {
        if (r - l == 1) { t[v] = a[l]; return; }
        int m = (l + r) / 2;
        build(2 * v, l, m, a);
        build(2 * v + 1, m, r, a);
        t[v] = t[2 * v] + t[2 * v + 1];
    }
    void apply(int v, int l, int r, ll x) { // apply +x to whole segment
        t[v] += x * (r - l);
        lz[v] += x;
    }
    void push(int v, int l, int r) {
        if (lz[v]) {
            int m = (l + r) / 2;
            apply(2 * v, l, m, lz[v]);
            apply(2 * v + 1, m, r, lz[v]);
            lz[v] = 0;
        }
```

```
    }
    void update(int v, int l, int r, int ql, int qr, ll x) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) { apply(v, l, r, x); return; }
        push(v, l, r);
        int m = (l + r) / 2;
        update(2 * v, l, m, ql, qr, x);
        update(2 * v + 1, m, r, ql, qr, x);
        t[v] = t[2 * v] + t[2 * v + 1];
    }
    ll query(int v, int l, int r, int ql, int qr) {
        if (qr <= l || r <= ql) return 0; // identity
        if (ql <= l && r <= qr) return t[v];
        push(v, l, r);
        int m = (l + r) / 2;
        return query(2 * v, l, m, ql, qr) + query(2 * v + 1, m, r, ql, qr);
    }
    void update(int l, int r, ll x) { update(1, 0, n, l, r, x); } // a[l..r) += x
    ll query(int l, int r) { return query(1, 0, n, l, r); }  // sum a[l..r)
};
```

Graphs

Bellman Ford

Time Complexity: $O(N.M)$ **Space Complexity:** $O(N)$

```
for (int i = 1; i <= n; ++i) {
    for (auto [from, to, score] : edges) {
        if(distance[from] == INF) continue;
        ll new_distance = min(distance[to], distance[from] + score);
        if (i == n && new_distance < distance[to]) {
            cout << -1 << '\n'; // Negative Cycle
            return 0;
        }
        distance[to] = new_distance;
    }
}
cout << distance[n] << '\n';
```

Floyd Cycle

Time Complexity: $O(N)$ **Space Complexity:** $O(N)$

```
vector<int> extract_cycle(int x) {
    int a = x;
    int b = teleport[x];
    while (a != b) {
        a = teleport[a];
        b = teleport[teleport[b]];
    }
    vector<int> cycle;
    do {
        a = teleport[a];
        cycle.push_back(a);
    } while (a != b);
    return cycle;
}
```

SCC

Time Complexity: $O(N+M)$ **Space Complexity:** $O(N)$

```
vector<int> st;  // - stack holding the unclaimed vertices
vector<int> roots; // - keeps track of the SCC roots of the vertices
int timer;       // - dfs timestamp counter
vector<int> t_in; // - keeps track of the dfs timestamp of the vertices
vector<int> t_low; // - keeps track of the lowest t_in of unclaimed vertices
                   // reachable in the subtree

// implements the tarjan algorithm for strongly connected components
void dfs(int v, vector<vector<int>> const &adj, vector<vector<int>> &components) {

  t_low[v] = t_in[v] = timer++;
  st.push_back(v);

  for (auto u : adj[v]) {
    if (t_in[u] == -1) { // tree-edge
      dfs(u, adj, components);
```

```
      t_low[v] = min(t_low[v], t_low[u]);
    } else if (roots[u] == -1) { // back-edge, cross-edge or forward-edge to an unclaimed vertex
      t_low[v] = min(t_low[v], t_in[u]);
    }
  }

  if (t_low[v] == t_in[v]) { // vertex is a root
    components.push_back({v}); // initializes a new component with root v
    while (true) {
      int u = st.back();
      st.pop_back();
      roots[u] = v; // claims the vertex
      if (u == v)
        break;
      components.back().push_back(u); // adds vertex u to the component of v
    }
  }
}

// input: adj -- adjacency list of G
// output: components -- the strongy connected components in G
// output: adj_cond -- adjacency list of G^SCC (by root vertices)
void strongly_connected_components(vector<vector<int>> const &adj,
                                   vector<vector<int>> &components,
                                   vector<vector<int>> &adj_cond) {
  components.clear();
  adj_cond.clear();

  int n = adj.size();

  st.clear();
  roots.assign(n, -1);
  timer = 0;
  t_in.assign(n, -1);
  t_low.assign(n, -1);

  // applies the tarjan algorithm to all the vertices
  // adds vertices to the components in reverse topological order
  for (int v = 0; v < n; v++) {
    if (t_in[v] == -1) {
      dfs(v, adj, components);
    }
  }

  // adds edges to the condensation graph
  adj_cond.assign(n, {});
  for (int v = 0; v < n; v++) {
    for (auto u : adj[v])
      if (roots[v] != roots[u])
        adj_cond[roots[v]].push_back(roots[u]);
  }
}
```

Bridges
Time Complexity: $O(N + M)$ **Space Complexity:** $O(N)$

```
void IS_BRIDGE(int v,int to); // some function to process the found bridge
int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    bool parent_skipped = false;
    for (int to : adj[v]) {
        if (to == p && !parent_skipped) {
            parent_skipped = true;
            continue;
        }
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
```

```
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v])
                IS_BRIDGE(v, to);
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}
```

Articulation Points

Time Complexity: $O(N+M)$ **Space Complexity:** $O(N)$

```
int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children=0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p!=-1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if(p == -1 && children > 1)
        IS_CUTPOINT(v);
}

void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs (i);
    }
}
```

Trees

CentroidAndHLD

Time Complexity: $O(N+M)$ **Space Complexity:** $O(N)$

```
#include <bits/stdc++.h>
using namespace std;

/* ============================================================================
 *  CENTROID DECOMPOSITION                              O(N log N) build
 * ----------------------------------------------------------------------------
 *  Idea: a centroid is a vertex whose removal leaves components of size <= N/2.
 *  Remove it, recurse on each component. The centroids form a "centroid tree"
 *  of depth O(log N). Any path u-v in the original tree passes through the
 *  centroid-tree LCA of u and v -> "paths through a vertex" divide & conquer.
 *
```

```
 *  Uses: count/aggregate paths with a property (length == K, sum <= K, ...),
 *        nearest marked vertex queries with updates, dist-based queries
 *        (sum of dists to marked set), any "for every path..." counting.
 *
 *  Everything is iterative (no recursion depth problems), 0-indexed.
 *
 *  Provided data after build():
 *    cpar[v]      parent of v in centroid tree (-1 for root centroid)
 *    clevel[v]    depth of v in centroid tree (root centroid = 0)
 *    distAnc[v][k] distance in ORIGINAL tree from v to its centroid ancestor
 *                 at level k (k = 0..clevel[v]; distAnc[v][clevel[v]] = 0).
 *                 Memory N log N; delete if you don't need it.
 * ========================================================================== */
struct CentroidDecomposition {
    int n;
    vector<vector<int>> g;
    vector<int> cpar, clevel, sz, tmpPar, tmpDist, bfs;
    vector<char> dead;              // dead[v] = v already chosen as centroid
    vector<vector<int>> distAnc;

    explicit CentroidDecomposition(int n)
        : n(n), g(n), cpar(n, -1), clevel(n, 0), sz(n), tmpPar(n),
          tmpDist(n), dead(n, 0), distAnc(n) {}

    void addEdge(int u, int v) { g[u].push_back(v); g[v].push_back(u); }

    // Centroid of the (alive) component containing `root`.
    int findCentroid(int root) {
        bfs.clear(); bfs.push_back(root); tmpPar[root] = -1;
        for (size_t i = 0; i < bfs.size(); i++) { // BFS order of component
            int u = bfs[i]; sz[u] = 1;
            for (int v : g[u])
                if (v != tmpPar[u] && !dead[v]) { tmpPar[v] = u; bfs.push_back(v); }
        }
        for (int i = (int)bfs.size() - 1; i > 0; i--) // subtree sizes bottom-up
            sz[tmpPar[bfs[i]]] += sz[bfs[i]];
        int total = bfs.size(), c = root;
        while (true) {                            // walk towards heavy child
            bool moved = false;
            for (int v : g[c])
                if (v != tmpPar[c] && !dead[v] && 2 * sz[v] > total) { c = v; moved = true; break; }
            if (!moved) return c;
        }
    }

    // process(c) is called once per centroid c, BEFORE c is removed:
    //   - the alive component of c (all vertices with !dead[]) is exactly the
    //     part of the tree this centroid is responsible for,
    //   - c itself is still alive, so DFS/BFS from c covers the whole component.
    // Typical process(): DFS from c, collect (depth, value) of each vertex,
    // combine, and subtract same-child-subtree contributions (inclusion-exclusion).
    template <class F> void build(F process) {
        vector<array<int, 3>> st;                 // {root of comp, centroid parent, level}
        st.push_back({0, -1, 0});
        while (!st.empty()) {
            auto [r, p, lvl] = st.back(); st.pop_back();
            int c = findCentroid(r);
            cpar[c] = p; clevel[c] = lvl;

            bfs.clear(); bfs.push_back(c); tmpPar[c] = -1; tmpDist[c] = 0;
            for (size_t i = 0; i < bfs.size(); i++) { // dist from c to its component
                int u = bfs[i];
                distAnc[u].push_back(tmpDist[u]);   // index == lvl (top-down order)
                for (int v : g[u])
                    if (v != tmpPar[u] && !dead[v]) { tmpPar[v] = u; tmpDist[v] = tmpDist[u] + 1; bfs.push_back(v); }
            }

            process(c);
            dead[c] = 1;
            for (int v : g[c]) if (!dead[v]) st.push_back({v, c, lvl + 1});
        }
    }
    void build() { build([](int) {}); }           // just build the centroid tree
};

/* --- EXAMPLE: nearest marked vertex, mark(v) / query(v) in O(log N) ----------
```

```
 *   CentroidDecomposition cd(n); (addEdge...) cd.build();
 *   const int INF = 1e9;
 *   vector<int> best(n, INF); // best[a] = min dist from centroid a to a marked vertex in its comp
 *   auto mark = [&](int v) {
 *       for (int a = v; a != -1; a = cd.cpar[a])
 *           best[a] = min(best[a], cd.distAnc[v][cd.clevel[a]]);
 *   };
 *   auto query = [&](int v) {
 *       int res = INF;
 *       for (int a = v; a != -1; a = cd.cpar[a])
 *           res = min(res, best[a] + cd.distAnc[v][cd.clevel[a]]);
 *       return res;
 *   };
 *
 * --- EXAMPLE: count paths of length exactly K (process(c) skeleton) ---------
 *   cnt[d] = number of vertices at depth d seen so far from c (cnt[0] = 1 for c).
 *   for each child v of c (alive): DFS v collecting depths list L;
 *       ans += sum over d in L of cnt[K - d]; // pair with previous subtrees
 *       then add L into cnt.                  // so same-subtree pairs not counted
 *   clear cnt afterwards (only up to max depth, to keep O(size) per centroid).
 * ------------------------------------------------------------------------ */


/* ========================================================================
 *  HEAVY-LIGHT DECOMPOSITION               O(N) build, O(log N) LCA,
 *                                          O(log^2 N) path query w/ seg tree
 * ------------------------------------------------------------------------
 *  Idea: heavy child = child with the largest subtree. Heavy edges form
 *  vertical chains. Any root->v path crosses only O(log N) light edges, hence
 *  touches O(log N) chains. If positions are assigned by DFS visiting the
 *  heavy child FIRST, then
 *      - every chain is a contiguous range [pos[head], pos[bottom]]
 *      - every subtree is a contiguous range [pos[v], pos[v] + sz[v] - 1]
 *  so a path = O(log N) ranges, a subtree = 1 range -> use segment tree / BIT.
 *
 *  Uses: path sum/max/min/gcd with point or range updates, subtree queries,
 *        LCA without binary lifting, edge-weighted path queries,
 *        "add x on path, query on path/subtree" problems.
 *
 *  Iterative (no recursion), 0-indexed. Works on a forest if you call build()
 *  per root (pos continues from `timer`) - for a single tree just build(0).
 * ======================================================================== */
struct HLD {
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> par, depth, sz, heavy, head, pos, vertAt; // vertAt[pos] = vertex

    explicit HLD(int n)
        : n(n), g(n), par(n, -1), depth(n, 0), sz(n, 1), heavy(n, -1),
          head(n, 0), pos(n, 0), vertAt(n, 0) {}

    void addEdge(int u, int v) { g[u].push_back(v); g[v].push_back(u); }

    void build(int root = 0) {
        // 1) BFS order -> parent, depth
        vector<int> order; order.reserve(n);
        order.push_back(root); par[root] = -1; depth[root] = 0;
        for (size_t i = 0; i < order.size(); i++) {
            int u = order[i];
            for (int v : g[u])
                if (v != par[u]) { par[v] = u; depth[v] = depth[u] + 1; order.push_back(v); }
        }
        // 2) sizes + heavy child, bottom-up
        for (int i = (int)order.size() - 1; i > 0; i--) {
            int v = order[i], p = par[v];
            sz[p] += sz[v];
        }
        for (int u : order) {
            int best = 0;
            for (int v : g[u])
                if (v != par[u] && sz[v] > best) { best = sz[v]; heavy[u] = v; }
        }
        // 3) preorder with heavy child visited first (pushed LAST on the stack)
        vector<int> st = {root}; head[root] = root;
        while (!st.empty()) {
            int u = st.back(); st.pop_back();
```

```
            pos[u] = timer; vertAt[timer++] = u;
            for (int v : g[u])
                if (v != par[u] && v != heavy[u]) { head[v] = v; st.push_back(v); }
            if (heavy[u] != -1) { head[heavy[u]] = head[u]; st.push_back(heavy[u]); }
        }
    }

    int lca(int u, int v) const {
        while (head[u] != head[v]) {
            if (depth[head[u]] < depth[head[v]]) swap(u, v);
            u = par[head[u]];
        }
        return depth[u] < depth[v] ? u : v;
    }
    int dist(int u, int v) const { return depth[u] + depth[v] - 2 * depth[lca(u, v)]; }

    // Calls f(l, r) on O(log N) inclusive position ranges covering path u..v.
    //   edges = false: values live on vertices.
    //   edges = true : value of edge (child, parent) is stored at pos[child];
    //                  the LCA's position is excluded.
    // NOTE: ranges come in arbitrary order (not u->v). For NON-commutative
    // operations collect the segments and orient them yourself.
    template <class F> void processPath(int u, int v, F f, bool edges = false) const {
        while (head[u] != head[v]) {
            if (depth[head[u]] < depth[head[v]]) swap(u, v);
            f(pos[head[u]], pos[u]);
            u = par[head[u]];
        }
        if (depth[u] > depth[v]) swap(u, v);
        if (pos[u] + edges <= pos[v]) f(pos[u] + edges, pos[v]);
    }

    // Subtree of v is the single range [pos[v], pos[v] + sz[v] - 1]
    template <class F> void processSubtree(int v, F f) const {
        f(pos[v], pos[v] + sz[v] - 1);
    }
};

/* --- Minimal segment tree to pair with HLD (point update, range sum) -------
 *  Swap `+` / 0 for max / min / gcd as needed. Half-open [l, r) internally. */
struct SegTree {
    int n; vector<long long> t;
    explicit SegTree(int n) : n(n), t(2 * n, 0) {}
    void set(int p, long long x) { for (t[p += n] = x; p > 1; p >>= 1) t[p >> 1] = t[p] + t[p ^ 1]; }
    long long query(int l, int r) {            // inclusive [l, r]
        long long res = 0;
        for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
            if (l & 1) res += t[l++];
            if (r & 1) res += t[--r];
        }
        return res;
    }
};

/* --- EXAMPLE: vertex values, path sum + point update + subtree sum ----------
 *   HLD h(n); (addEdge...); h.build(0);
 *   SegTree st(n);
 *   for (int v = 0; v < n; v++) st.set(h.pos[v], a[v]);
 *   // update vertex v to x:
 *   st.set(h.pos[v], x);
 *   // path sum u..v:
 *   long long s = 0; h.processPath(u, v, [&](int l, int r) { s += st.query(l, r); });
 *   // subtree sum of v:
 *   long long t = 0; h.processSubtree(v, [&](int l, int r) { t += st.query(l, r); });
 *
 * --- EXAMPLE: edge weights: for edge (p, c) with c the deeper endpoint
 *   st.set(h.pos[c], w); and query with processPath(u, v, f, true) [true = edges mode].
 * ------------------------------------------------------------------------ */
```